

Lab 6 Report: Efficient and Modular Path Planning, and Trajectory Following, and Localization for Autonomous Navigation

Team 1

Shrika Eddula
Sammy Krem
Pyae Sone Nyo Hmine
Sydney Matthews
Jayson Lin

6.4200

April 24, 2025

1 Introduction: Shrika Eddula

Autonomous robot navigation is a critical capability for various robotics applications, enabling robots to move efficiently and safely through environments without human intervention. Successful navigation relies on three fundamental capabilities: localization, path planning, and trajectory control. Localization estimates the robot's position in space, path planning computes feasible routes, and trajectory control ensures the robot follows those routes accurately.

In this lab, we built on our prior implementation of Monte Carlo Localization (MCL), aiming to create a complete autonomous navigation system. The primary challenge of this lab was to integrate these essential components—localization, path planning, and control—into a cohesive system that could navigate in both simulated and real-world environments.

We implemented two path planning strategies to address different navigation needs. The first is a search-based algorithm (A*) that works in discrete grid space, ideal for environments with well-defined obstacles. The second is a sampling-based algorithm (Rapidly-exploring Random Tree, RRT), designed for continuous spaces and better suited for more dynamic and less structured environments. Both algorithms operate on occupancy grid maps, incorporating dilated obstacles to account for the robot's physical size. Additionally, cubic spline-based smoothing is optionally applied to refine paths, ensuring they align with the robot's motion constraints.

For the control aspect, we used a pure pursuit algorithm, which balances simplicity with effective handling of non-holonomic constraints. This algorithm continuously selects a lookahead target point along the path and computes the necessary steering commands to guide the robot.

We utilized ROS2's architecture to seamlessly integrate these components, enabling the robot to autonomously navigate from its current location—estimated by MCL—to user-specified goals while avoiding obstacles along the way. The lab also includes real-world testing to assess system performance in dynamic environments, with a focus on metrics like path time, tracking accuracy, and overall navigation success.

2 Technical Implementation: Shrika Eddula

The core challenge of this lab was to design a path planner capable of planning a path and driving to its goal. Building on our previous localization work, we implemented and integrated path planning and trajectory following to create a complete autonomous navigation system using ROS2, a robotic software framework. This section presents our technical implementation, with a focus on the new components and their integration with our existing localization framework.

2.0.1 Path Planning

Our path planning system generates collision-free trajectories from the robot’s current position (from the localization system) to a specified goal. We implemented two planning approaches, each with distinct strengths: a search-based algorithm (A*) and a sampling-based algorithm (RRT).

Map Preprocessing Before planning, we preprocess the occupancy grid map using morphological dilation to account for the robot’s dimensions. This begins by converting the grid into a binary map (0 for free, 1 for occupied), followed by convolution with a rectangular structuring kernel of configurable size. The dilation expands obstacles outward, creating buffer zones with values of 100 in the occupancy grid. This avoids planning paths too close to obstacles, with the dilation parameter adjusting safety margins based on robot width and localization uncertainty—larger values for higher error, smaller for tight areas.

Search-Based Planning (A*) For our search-based planning approach, we implemented the A* algorithm, which finds the optimal path by exploring the grid space while using a heuristic to guide the search toward the goal. The algorithm maintains a priority queue of nodes to be explored, where the priority of each node n is determined by the function:

$$f(n) = g(n) + \epsilon \cdot h(n) \quad (1)$$

where $g(n)$ is the cost accumulated from the start node to n , $h(n)$ is a heuristic estimate of the cost from n to the goal, and $\epsilon \geq 1$ is a weighting factor. In our implementation, $\epsilon = 1.001$, providing a slight bias toward more direct paths.

The A* algorithm can be formalized as follows:

1. Initialize the open set with the start node s , with $g(s) = 0$ and $f(s) = h(s)$
2. While the open set is not empty:
 - Remove the node n with lowest $f(n)$ from the open set
 - If n is the goal node, reconstruct and return the path
 - For each neighbor m of n :
 - Calculate tentative cost: $g_{temp}(m) = g(n) + c(n, m)$
 - If m is not yet discovered or $g_{temp}(m) < g(m)$:
 - * Update $g(m) = g_{temp}(m)$
 - * Set predecessor of m to n
 - * Calculate $f(m) = g(m) + \epsilon \cdot h(m)$
 - * Add m to the open set
3. Return failure if open set becomes empty

Our heuristic function $h(n)$ uses the Euclidean distance:

$$h(n) = \sqrt{(n_x - goal_x)^2 + (n_y - goal_y)^2} \quad (2)$$

This heuristic is admissible, ensuring that A* finds the optimal path when $\epsilon = 1$.

For the cost function $c(n, m)$ between adjacent nodes, we assign different weights based on the movement type:

$$c(n, m) = \begin{cases} 1.0 & \text{if movement is orthogonal} \\ \sqrt{2} \approx 1.414 & \text{if movement is diagonal} \end{cases} \quad (3)$$

When generating neighboring nodes, our implementation considers all eight adjacent grid cells in the set $\mathcal{N}(n)$:

$$\mathcal{N}(n) = \{(n_x + dx, n_y + dy) \mid dx, dy \in \{-1, 0, 1\}, (dx, dy) \neq (0, 0)\} \quad (4)$$

For diagonal movements, we add an additional constraint to prevent corner-cutting through obstacles. A diagonal move from n to m is allowed only if both orthogonal cells adjacent to the diagonal are also free:

$$\text{valid_diagonal}(n, m) \iff \text{is_free}(n_x, m_y) \wedge \text{is_free}(m_x, n_y) \quad (5)$$

Once the goal is reached, we reconstruct the path by tracing back from the goal to the start using the predecessor pointers. The resulting path $P = \{p_1, p_2, \dots, p_k\}$ consists of a sequence of grid cells, where p_1 is the start and p_k is the goal.

Sampling-Based Planning (RRT) Our sampling-based planning implementation uses the Rapidly-exploring Random Tree (RRT) algorithm, which builds a tree of reachable states by incrementally growing from the start position toward randomly sampled points in the environment. The algorithm can be formally described as follows:

1. Initialize tree $\mathcal{T}$ with the start node: $\mathcal{T}.V = \{x_{start}\}$, $\mathcal{T}.E = \emptyset$
2. For $i = 1$ to N iterations:
 - With probability p_{goal} , set $x_{rand} = x_{goal}$
 - Otherwise, sample x_{rand} randomly from the free configuration space $\mathcal{X}_{free}$
 - Find nearest node in tree: $x_{nearest} = \underset{x \in \mathcal{T}.V}{\text{argmin}} d(x, x_{rand})$
 - Create new node by steering from $x_{nearest}$ toward x_{rand} :

$$x_{new} = \text{steer}(x_{nearest}, x_{rand}, \Delta q) \quad (6)$$

- If the path from $x_{nearest}$ to x_{new} is collision-free:
 - Add x_{new} to the tree: $\mathcal{T}.V = \mathcal{T}.V \cup \{x_{new}\}$
 - Add edge to the tree: $\mathcal{T}.E = \mathcal{T}.E \cup \{(x_{nearest}, x_{new})\}$
 - If $d(x_{new}, x_{goal}) < \epsilon_{goal}$, terminate and return path

3. Extract path by tracing from x_{goal} to x_{start} through parent links

In our implementation, we use $p_{goal} = 0.1$ (10% probability of sampling the goal directly) and distance function $d(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$ (Euclidean distance).

The *steer* function limits the distance of growth to a maximum step size Δq , ensuring the tree expands in a controlled manner:

$$\text{steer}(x_1, x_2, \Delta q) = \begin{cases} x_2 & \text{if } d(x_1, x_2) \leq \Delta q \\ x_1 + \Delta q \cdot \frac{x_2 - x_1}{d(x_1, x_2)} & \text{otherwise} \end{cases} \quad (7)$$

For collision checking, we verify both that the new node itself is in free space and that the path segment connecting $x_{nearest}$ to x_{new} is collision-free. The latter check uses linear interpolation to sample k points along the segment, where k is proportional to the segment length:

$$\text{collision_free}(x_1, x_2) \iff \forall t \in \{0, \frac{1}{k}, \frac{2}{k}, \dots, 1\} : \text{is_free}(x_1 + t(x_2 - x_1)) \quad (8)$$

Once the goal region is reached (a node comes within ϵ_{goal} distance of the goal), we reconstruct the path by tracing back through the tree from the goal node to the start node using the parent-child relationships stored in the edges $\mathcal{T}.E$.

The RRT algorithm’s probabilistic completeness guarantees that as the number of samples approaches infinity, the probability of finding a path (if one exists) approaches 1. However, the basic RRT does not guarantee optimality. The algorithm’s efficiency comes from its ability to rapidly explore the configuration space without requiring explicit discretization, making it particularly suitable for planning in continuous domains.

Path Smoothing We implemented optional spline-based path smoothing to create more natural trajectories. After generating the initial path with A* or RRT, cubic spline interpolation fits piecewise polynomials through the waypoints, creating continuous derivatives throughout the path. This reduces sharp turns and angular discontinuities common in grid-based plans, producing trajectories that better align with the vehicle’s kinematic constraints and are easier for the pure pursuit controller to follow accurately. Our implementation balances smoothness and safety by keeping the spline close to original waypoints while eliminating severe discontinuities. This feature can be enabled or disabled through a parameter to suit different navigation scenarios.

2.0.2 Pure Pursuit Controller

To follow planned paths, we implemented a pure pursuit controller that continually selects a “lookahead” target point on the trajectory and computes steering commands to reach it. This geometric method yields smooth, kinematically feasible control.

Lookahead Point Selection We find the lookahead point in two steps. First, we locate the point on the polyline path closest to the vehicle by projecting its current position P onto each segment $[A, B]$. For each segment:

$$t = \frac{(P - A) \cdot (B - A)}{(B - A) \cdot (B - A)}, \quad t \leftarrow \min(\max(t, 0), 1), \quad P_{\text{proj}} = A + t(B - A),$$

and choose the projection with the smallest distance to P .

Second, starting from that segment, we search forward for an intersection with the circle centered at P of radius L_d . For a segment $[A, B]$ and radius r :

$$a = (B - A) \cdot (B - A), \quad b = 2(A - P) \cdot (B - A), \quad c = (A - P) \cdot (A - P) - r^2,$$

$$\Delta = b^2 - 4ac, \quad t = \frac{-b \pm \sqrt{\Delta}}{2a},$$

clamping each t to $[0, 1]$. We pick the valid intersection with the largest t . If none exists, we use the final waypoint. This approach handles edge cases—off-path positions or sharp turns—gracefully.

Steering Angle Computation Once identified, the lookahead point $(x_{\text{la}}, y_{\text{la}})$ is transformed into the vehicle’s local frame via its global pose $(x_{\text{car}}, y_{\text{car}}, \psi)$:

$$\Delta x = x_{\text{la}} - x_{\text{car}}, \quad \Delta y = y_{\text{la}} - y_{\text{car}},$$

$$x_{\text{loc}} = \cos(-\psi) \Delta x - \sin(-\psi) \Delta y, \quad y_{\text{loc}} = \sin(-\psi) \Delta x + \cos(-\psi) \Delta y.$$

If $x_{\text{loc}} \leq 0$, the point lies behind the vehicle and we command zero steering to avoid increased deviation. Otherwise, we compute the heading error $\eta = \arctan(\frac{y_{\text{loc}}}{x_{\text{loc}}})$ and apply the bicycle-model curvature formula:

$$\delta = \arctan\left(\frac{2L \sin \eta}{L_d}\right),$$

where L is the wheelbase and L_d the lookahead distance. We saturate δ within the vehicle’s steering limits before issuing the command.

Parameter Tuning Two parameters dominate behavior:

- **Lookahead distance** L_d . Larger L_d yields smoother tracking but looser path adherence; smaller L_d tightens tracking at the cost of oscillations. We found $L_d = 1.0$ m balances smoothness and accuracy for our platform.
- **Speed** v . A constant 4.0 m/s was used in experiments; the controller can accept variable v profiles for future work.

The wheelbase $L = 0.34$ m is fixed by the robot’s geometry.

Visualization and Debugging We integrated RViz markers to visualize components in real time:

- A circle around the vehicle with radius L_d shows the search region for lookahead candidates.
- A line from the vehicle to the selected point confirms correct selection.
- The original and—when enabled—splined trajectories are drawn to assess smoothing and obstacle clearance.
- For RRT, we render the growing search tree (nodes and edges) to inspect exploration patterns and verify collision checks.

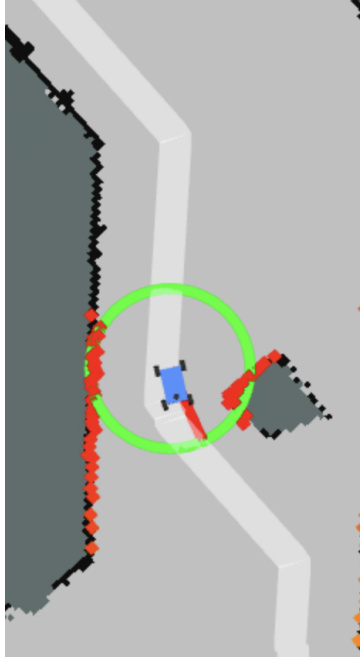


Figure 1: Lookahead search visualization



Figure 2: RRT search tree view

These visual cues proved invaluable for diagnosing off-path behavior, refining L_d , and tuning gains. Observing dynamic marker updates as the vehicle moves let us quickly catch and correct projection errors, mismatches, or steering saturation in diverse scenarios.

2.0.3 Integration with Localization

Our navigation system builds upon the Monte Carlo Localization (MCL) system developed previously. The localization component maintains a belief distribution of the robot's pose as weighted particles, updating it via motion predictions and measurement corrections from sensor data.

Monte Carlo Localization Overview The localization system uses a particle filter to estimate the robot's position and orientation in a known map. Each particle represents a possible robot pose as a three-dimensional vector (x, y, θ) corresponding to position and orientation in the map frame. The algorithm proceeds as follows:

1. **Initialization:** We distribute particles uniformly around an initial position estimate using controlled random spreads:

$$x_i = x_{init} + \mathcal{U}(-r_t, r_t) \quad (9)$$

$$y_i = y_{init} + \mathcal{U}(-r_t, r_t) \quad (10)$$

$$\theta_i = \theta_{init} + \mathcal{U}(-r_\theta, r_\theta) \quad (11)$$

where $\mathcal{U}(a, b)$ denotes a uniform random distribution with r_t and r_θ as translational and orientational spread radii.

2. **Motion Update:** When odometry data is received, we update each particle's position based on the robot's movement, transforming displacement from the robot frame to the world frame:

$$\Delta x_{world} = \Delta x \cdot \cos(\theta) - \Delta y \cdot \sin(\theta) \quad (12)$$

$$\Delta y_{world} = \Delta x \cdot \sin(\theta) + \Delta y \cdot \cos(\theta) \quad (13)$$

We then add Gaussian noise to model motion uncertainty:

$$x_{new} = x + \Delta x_{world} + \mathcal{N}(0, \sigma_x) \quad (14)$$

$$y_{new} = y + \Delta y_{world} + \mathcal{N}(0, \sigma_y) \quad (15)$$

$$\theta_{new} = \theta + \Delta \theta + \mathcal{N}(0, \sigma_\theta) \quad (16)$$

3. **Measurement Update:** When laser scan data is received, we evaluate how well each particle’s predicted measurements match observations. We use a beam-based sensor model combining four distributions to model LIDAR behavior:

$$P(z|d) = \alpha_{hit} \cdot P_{hit}(z|d) + \alpha_{short} \cdot P_{short}(z|d) + \alpha_{max} \cdot P_{max}(z|d) + \alpha_{rand} \cdot P_{rand}(z|d) \quad (17)$$

where z is the observed range, d the expected range, and the components model correct measurements with noise, unexpected obstacles, maximum range, and random noise.

4. **Resampling:** We select particles for the next iteration with probability proportional to their weights, focusing computation on promising regions of the state space.
5. **Pose Estimation:** We compute the estimated pose as a weighted average of particle positions:

$$\hat{x} = \sum_{i=1}^N w_i \cdot x_i \quad (18)$$

$$\hat{y} = \sum_{i=1}^N w_i \cdot y_i \quad (19)$$

For orientation, we use a circular mean to handle angle periodicity:

$$\hat{\theta} = \text{atan2} \left(\sum_{i=1}^N w_i \cdot \sin(\theta_i), \sum_{i=1}^N w_i \cdot \cos(\theta_i) \right) \quad (20)$$

The localization system publishes the estimated pose as both a transform between the map and particle filter frames, and as an Odometry message on `/pf/pose/odom`, providing crucial position data to the path planner.

Integration Pipeline Our localization system integrates with navigation components through a three-stage pipeline:

1. **Localization:** Particle filter estimates robot pose from sensor data, publishing to `/pf/pose/odom` for path following.
2. **Path Planning:** When a goal is specified, the planner retrieves current position, generates a collision-free path via A* or RRT using the map, optionally applies spline smoothing, and publishes to `/trajectory/current` as a PoseArray.
3. **Trajectory Following:** Pure pursuit controller subscribes to trajectory and particle filter odometry data, computes steering commands, and publishes drive commands.

This modular architecture enables independent component testing and flexible implementation swapping. ROS2’s publisher-subscriber model facilitates asynchronous communication with components running at different frequencies (trajectory follower at higher frequency). All calculations occur in the map frame with appropriate transformations to the robot’s local frame. Safety features include dilated obstacles, follower fallback behaviors, and a watchdog timer for emergency stops.

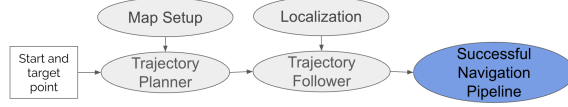


Figure 3: Navigation pipeline overview

2.1 Design Considerations

Efficiency-Accuracy Tradeoffs Optimized by using 200 particles, downsampling laser scans, pre-computing sensor model tables, implementing early termination conditions, and optimizing collision checking while maintaining real-time performance.

Robustness Incorporated thread synchronization, numerical stability safeguards, handling of edge cases (zero-weight particles, no-lookahead-point scenarios), and circular statistics for orientation calculations to prevent angular discontinuities.

Algorithmic Trade-offs Implemented both A* (optimal paths but computationally intensive) and RRT (faster planning with potentially suboptimal paths) to select appropriate algorithms based on specific navigation tasks.

Parameter Tuning Experimentally determined optimal values for motion and sensor models (localization), dilation size and spline smoothing (path planning), and lookahead distance and speed (trajectory following) to ensure reliable navigation across various environments.

3 Experimental Evaluation: Sammy Krem, Jayson Lin, Sydney Matthews, Pyae Sone Nyo Hmine

To facilitate easier integration, we tested the individual components of our pipeline: localization, path planning, and trajectory follower.

3.1 Localization MCL Algorithm on Different Hallways and Turns

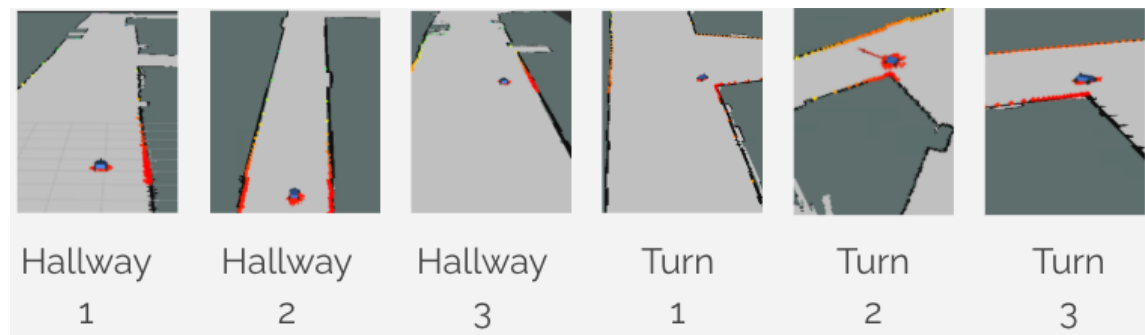


Figure 4: Hallways and turns used for our experiments, referenced in the experimental tables.

We tested the localization algorithm in simulation and real experiments across three hallways and turns. The wall follower was used for simulation, and teleoperation for the real robot. Success was defined as particle alignment with the robot at the end of its path, allowing for some drift. The

robot performed well in Hallways 1 and 3, but struggled in Hallway 2 due to fewer distinguishing features. Turn experiments performed better due to clearer localization markers.

3.2 Best Simulation Parameters for Strong Localization Performance

We iterated through various parameter combinations to find the best values, measuring the success rate at each step. The parameter set of (0.05, 0.05, 45) emerged as the best within simulation.

Table I: Strong Hallway Error (10 Trials)

	X error (m)	Y error (m)	Distance error (m)	\theta error (deg)
Hallway 1	0.318	0.035	0.326	1.20°
Hallway 2	0.589	0.043	0.592	1.71°
Hallway 3	0.065	0.323	0.329	2.79°
Average	0.324	0.134	0.415	1.90°

Table II: Strong Hallway Success

	Success % (10 trials)
Hallway 1	0.9
Hallway 2	0.8
Hallway 3	1.0
Average	0.9

Tables I and II show improved error and success rate. This set of standard deviations allowed the particles to explore enough of the state space without straying too far from the true pose, balancing between exploration and accuracy for stable localization.

Table III: Low Turn Errors (10 Trials)

	X error (m)	Y error (m)	Distance error (m)	\theta error (deg)
Turn 1	0.269	0.175	0.359	7.44°
Turn 2	0.154	0.2807	0.35	2.69°
Turn 3	0.184	0.165	0.281	3.45°
Average	0.202	0.207	0.33	4.52°

Table IV: Strong Turn Success (10 Trials)

	Success % (10 trials)
Turn 1	1.0
Turn 2	1.0
Turn 3	1.0
Average	1.0

Tables III and IV show that turning was 100% successful and precise, with minimal translational errors and no drift, due to the distinct geometry of turns allowing better localization. However, theta errors were larger as turns caused greater orientation changes.

3.3 Comparison between Path Planning Algorithms: Heuristic-based vs Sampling based

To evaluate the strengths and weaknesses of our path planning implementations, we compared the performance of the A* and RRT algorithms across three navigation scenarios in simulation. This comparison focused on two key metrics: **planning time** and **path length**, representing computational efficiency and trajectory optimality, respectively.

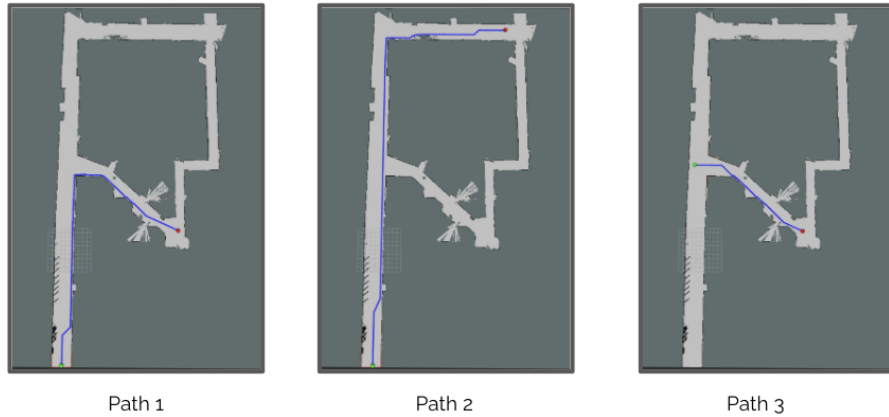
3.3.1 Qualitative Comparison

We observed consistent differences in path characteristics and behavior across all paths:

- **A* Planning:**
 - Generated smoother and shorter paths with fewer unnecessary turns.
 - Ideal for applications where path optimality is prioritized.
 - Tended to be more computationally intensive due to full map exploration.
- **RRT Planning:**
 - Much faster and more efficient in generating a path.
 - Well-suited for fast exploration and high-dimensional or complex environments.
 - Paths were typically longer, with occasional sharp turns due to the randomized nature of tree growth.
 - No guarantee of optimality due to sampling-based design.

The following figures show the three planned paths in simulation using A* and RRT:

A* Implementation in Sim



14

Figure 5: A* Implementation in Simulation for three path scenarios

RRT Implementation in Sim

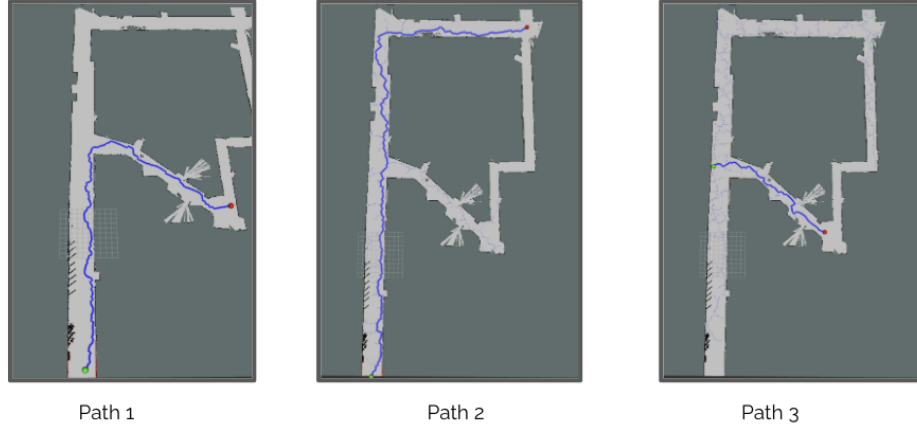


Figure 6: RRT Implementation in Simulation for three path scenarios

3.3.2 Quantitative Comparison: Planning Time and Path Length

We measured planning time and total path distance for both A* and RRT algorithms across three predefined start-goal pairs. The results are summarized below:

Table V: Planning Time Comparison (Lower is better)

Path	A* Time (s)	RRT Time (s)
Path 1	2.5415	0.1436
Path 2	4.4696	0.4197
Path 3	1.3976	1.1451

Table VI: Path Length Comparison (Lower is better)

Path	A* Distance (m)	RRT Distance (m)
Path 1	73.76	84.47
Path 2	109.93	132.06
Path 3	31.99	36.78

3.3.3 Conclusion of Comparison

The decision between A* and RRT depends heavily on the requirements of the navigation task:

- **Use A*** when accuracy, shortest path, and smoothness are critical.
- **Use RRT** when speed is essential, the environment is large or complex, or approximate solutions are acceptable.

This flexibility within our system allows us to switch planners based on the scenario, and even combine them with smoothing or hybrid strategies in future iterations.

3.4 Comparison between Regular and Splined Trajectories for both Path Planning Algorithms

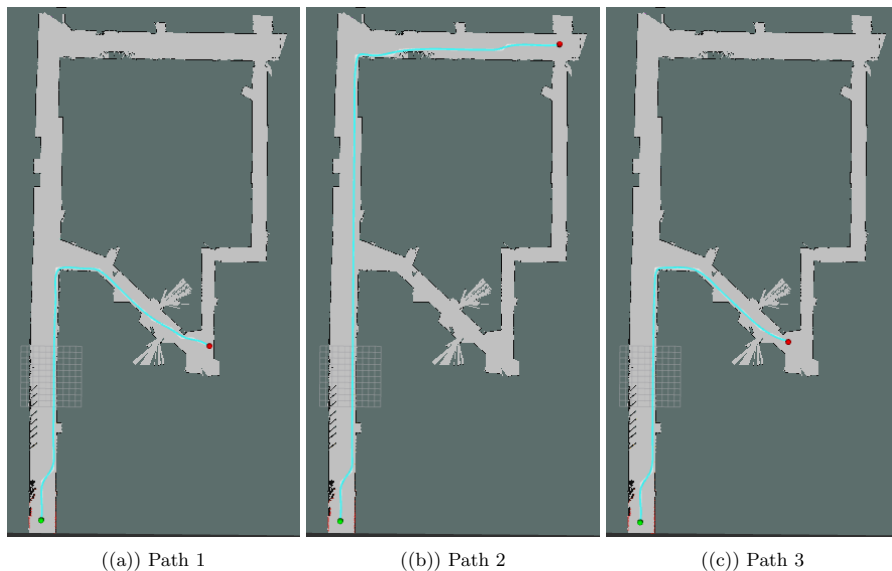


Figure 7: **A* paths (white) and splined versions (blue)**: The splined path is smoother and slightly shorter than the original.

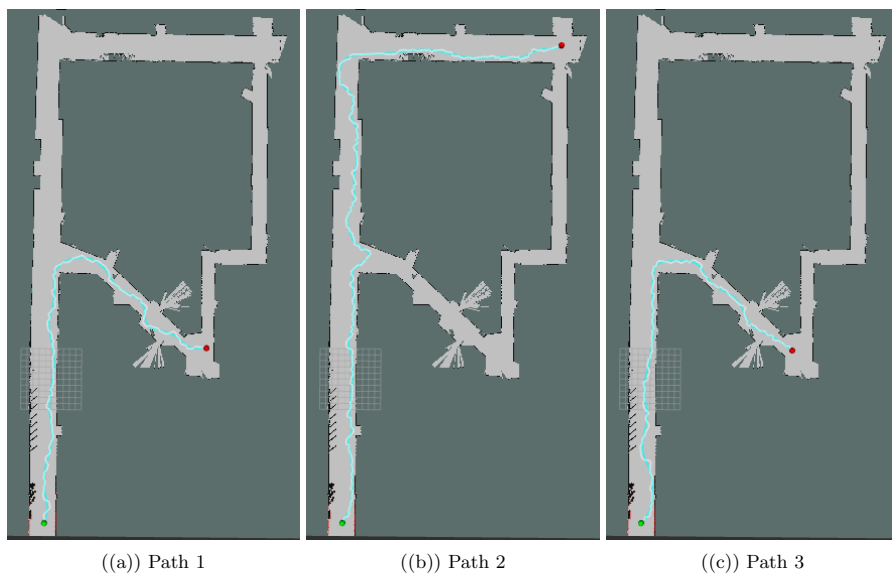


Figure 8: **RRT paths (white) and splined versions (blue)**: The splined RRT path is much smoother and shorter.

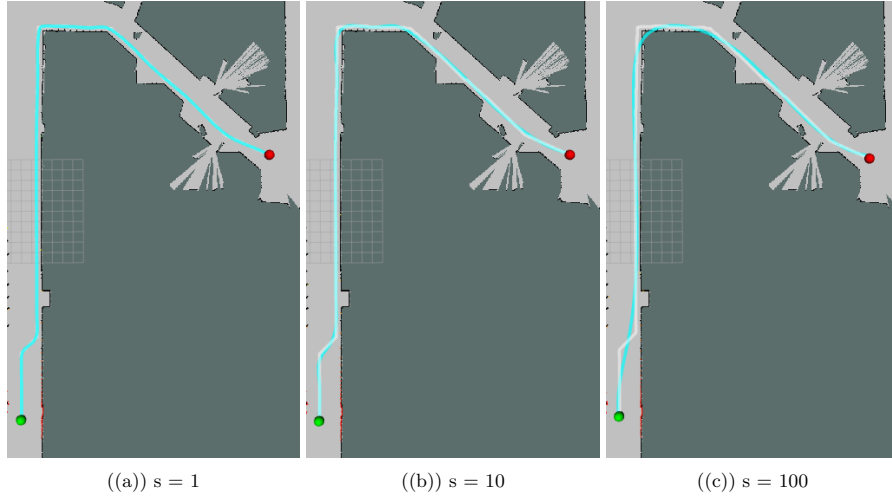


Figure 9: **Larger smoothing factor s leads to more deviation:** Smaller s values retain original path, while larger values smooth and diverge.

3.4.1 Experiment Setup

We applied both A* and RRT planners on the same test paths, comparing raw and splined outputs (Figures 7 and 8). Spline fitting is fast, so we focus on smoothness and total distance. All trials used identical start/end points in RVIZ, a fixed dilation factor, and smoothing parameter $s = 10$. Splines were generated with SciPy’s `splprep`, enforcing

$$\sum_{i=1}^n (y_i - g(u_i))^2 \leq s,$$

where y_i are original points, $g(u_i)$ the spline values, and s the residual threshold.

Table VII: Path Length Comparison (Lower is better)

Path	A* Splined Distance (m)	A* Regular Distance (m)
Path 1	71.08	73.05
Path 2	110.54	111.84
Path 3	68.09	69.81

Table VIII: Path Length Comparison (Lower is better)

Path	RRT Splined Distance (m)	RRT Regular Distance (m)
Path 1	74.7	83.89
Path 2	117.53	132.11
Path 3	72.06	81.69

3.4.2 Results

Splining smooths RRT paths and refines A* turns. The splined RRT paths are shorter, especially for complex paths, while A* shows a minor reduction. Smoother RRT trajectories reduce oscillations and distance; for A*, the impact is minimal due to its near-optimal layout.

3.5 Analysis of Pure Pursuit Trajectory Follower

We evaluated the pure pursuit controller’s performance on a test trajectory using both quantitative and qualitative metrics to analyze how lookahead distance and velocity affect tracking. Experiments were conducted in simulation with *RViz* to observe the robot’s movement.



Figure 10: Trajectory used with varied hallways, turns to evaluate controller performance under different velocities and lookahead distances.

To isolate the controller’s performance from localization noise, we used the ground truth base link pose for direct evaluation without sensor drift interference.

Simulation helped fine-tune parameters for robustness against sensor noise and dynamic obstacles. By varying lookahead distances and velocities, we assessed the controller’s ability to handle tight turns, sharp corners, and long hallways.

Testing Different Speeds at Constant Lookahead

We tested four velocities (1 m/s, 2 m/s, 3 m/s, 4 m/s) at a constant lookahead of 1 meter. Since the pure pursuit algorithm selects the next goal point based on distance, we expected minimal differences in the resulting path.

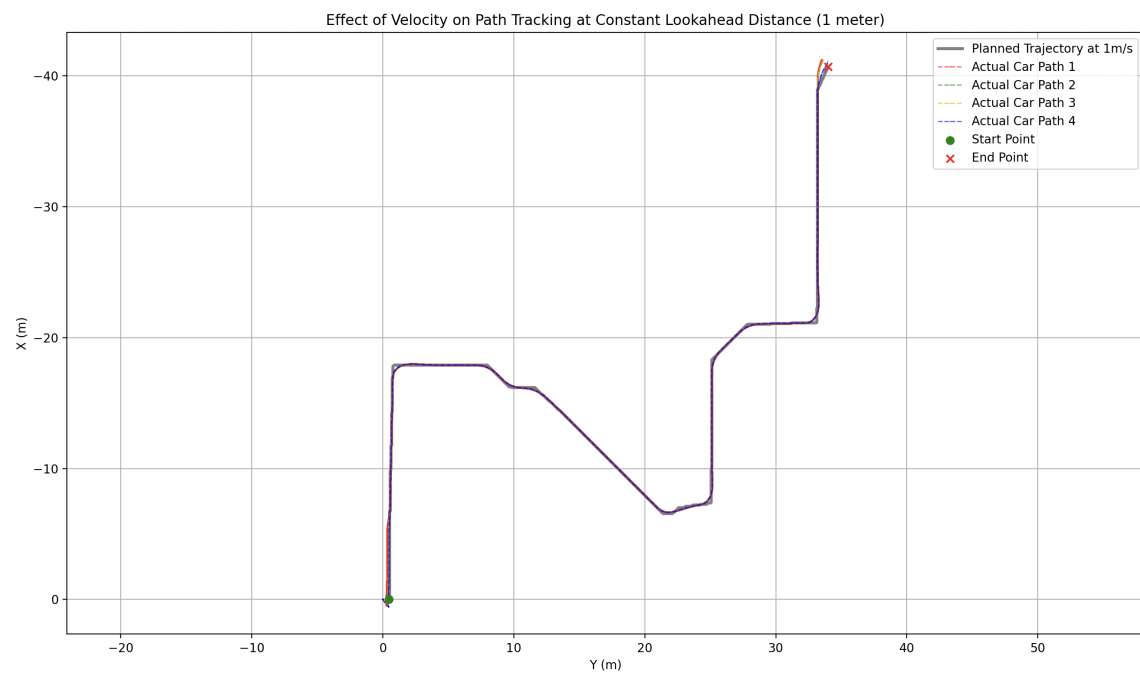


Figure 11: Trajectories generated using a constant lookahead of 1.0 m and varying velocities (1–4 m/s) appear nearly identical.

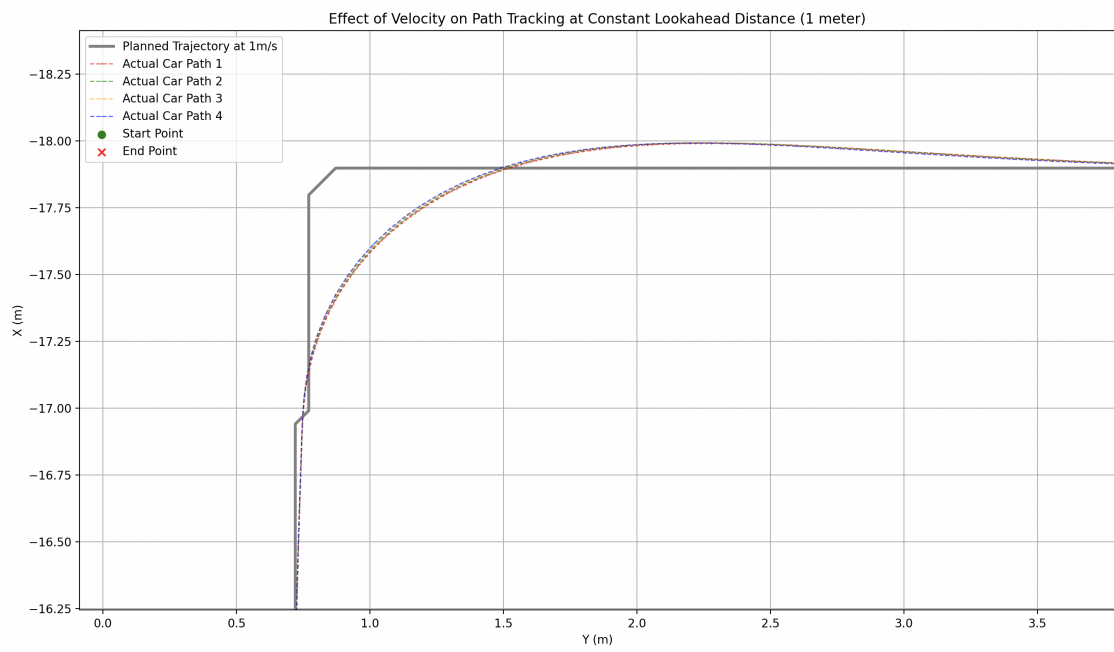
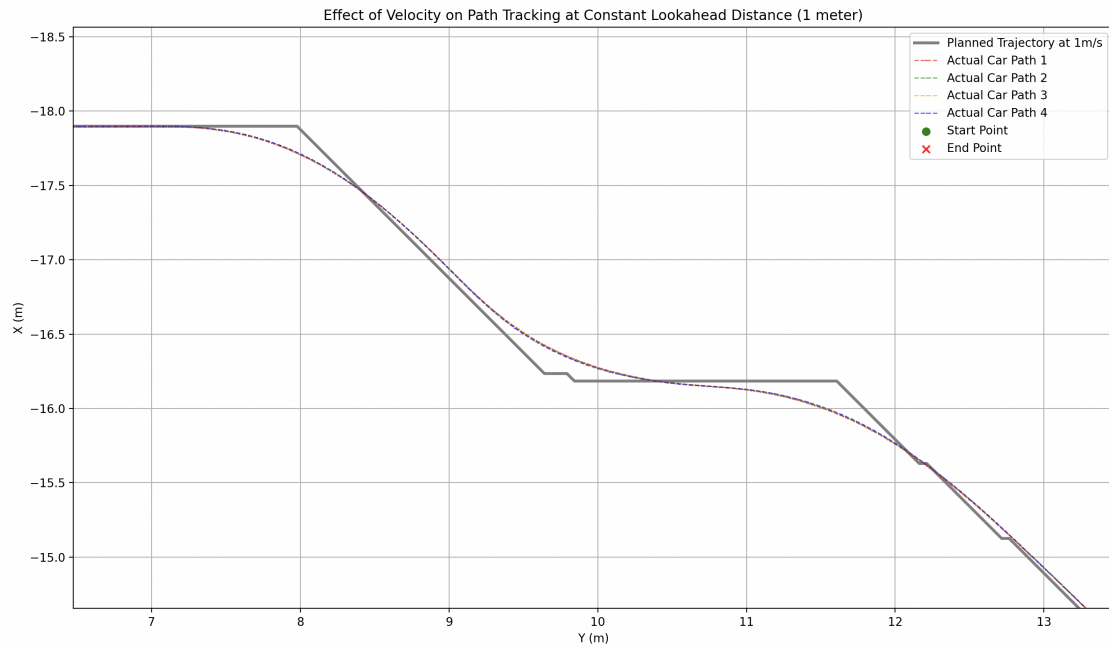


Figure 12: Different velocities don't affect minor adjustments or larger turns.

Table IX: Average Cross-track error metrics at constant lookahead (1.0 m) for varying speeds over 5 trials

Speed (m/s)	Mean Error (m)	Mean Absolute Error (m)	Standard Deviation (m)
1.0	-0.00025	0.02507	0.04816
2.0	-0.00012	0.02452	0.04699
3.0	-0.00118	0.02420	0.04688
4.0	-0.00003	0.02434	0.04599

Figure 11, Figure 12, and Table IX all confirmed that velocity does not significantly affect trajectory tracking with a fixed lookahead distance and known true pose.

Evaluating Lookahead Distances at Constant Velocity

To isolate the effect of lookahead distance, we fixed the vehicle speed at 4 m/s and tested four values: 0.5 m, 0.75 m, 1.0 m, and 1.25 m.

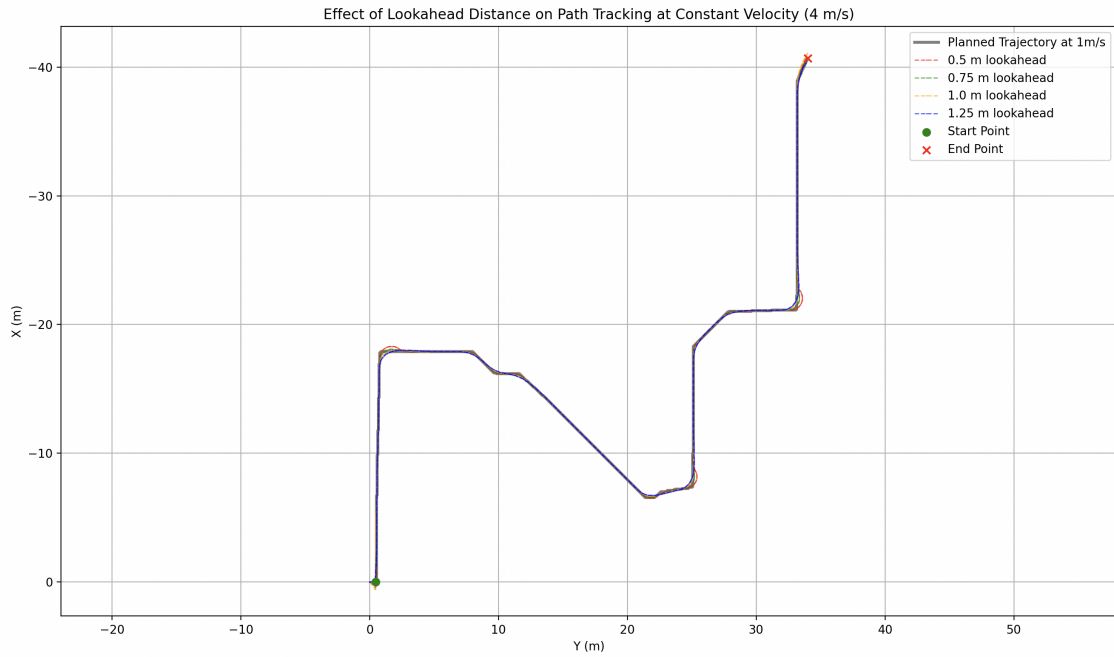


Figure 13: Paths at 4 m/s are different across different lookahead distances.

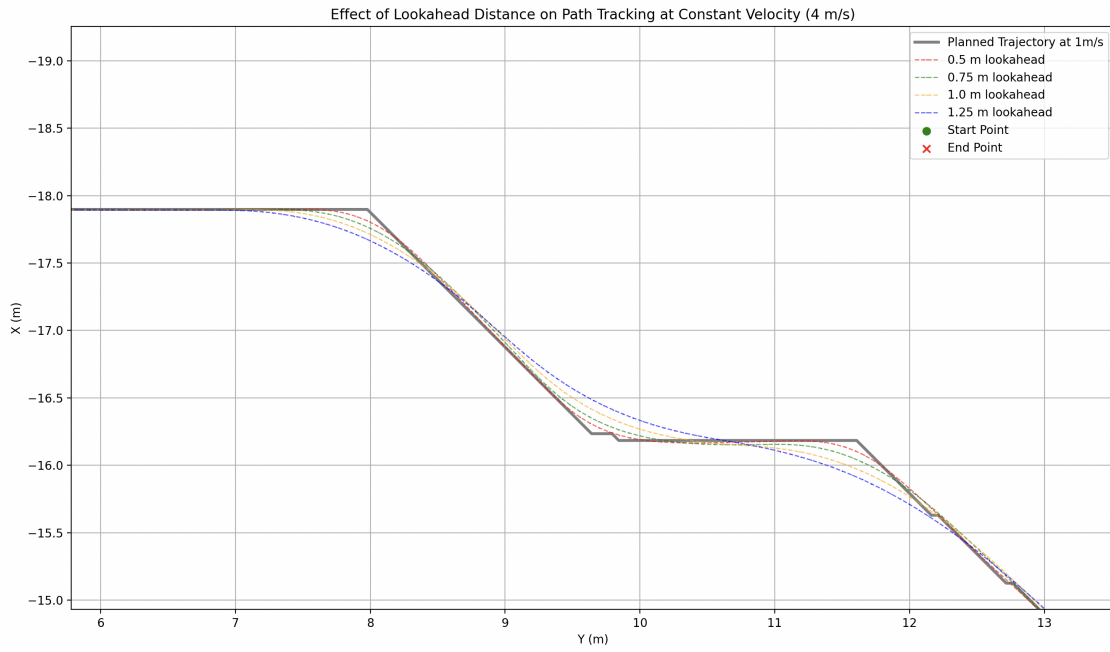


Figure 14: Larger lookaheads under-react to gentle curves.

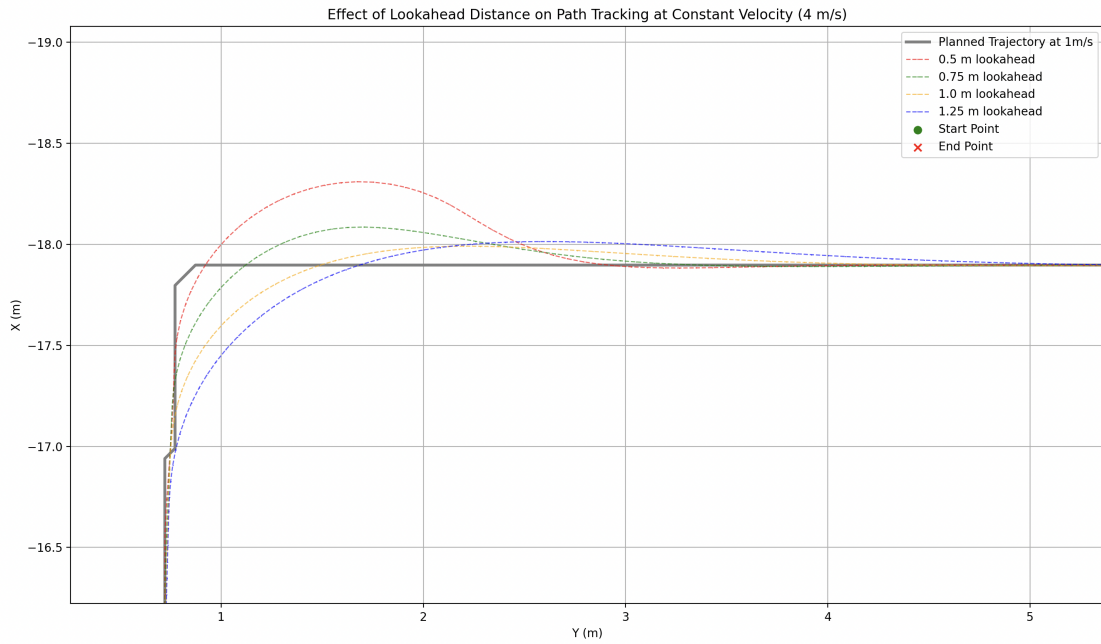


Figure 15: Short lookaheads overshoot; long lookaheads cut corners.

Figures 13, 14, and 15 show trends aligned with pure pursuit theory: longer lookaheads result in smoother, less precise turns, while shorter ones allow sharper but cause delays and oscillations. A medium-length lookahead balances stability and accuracy.

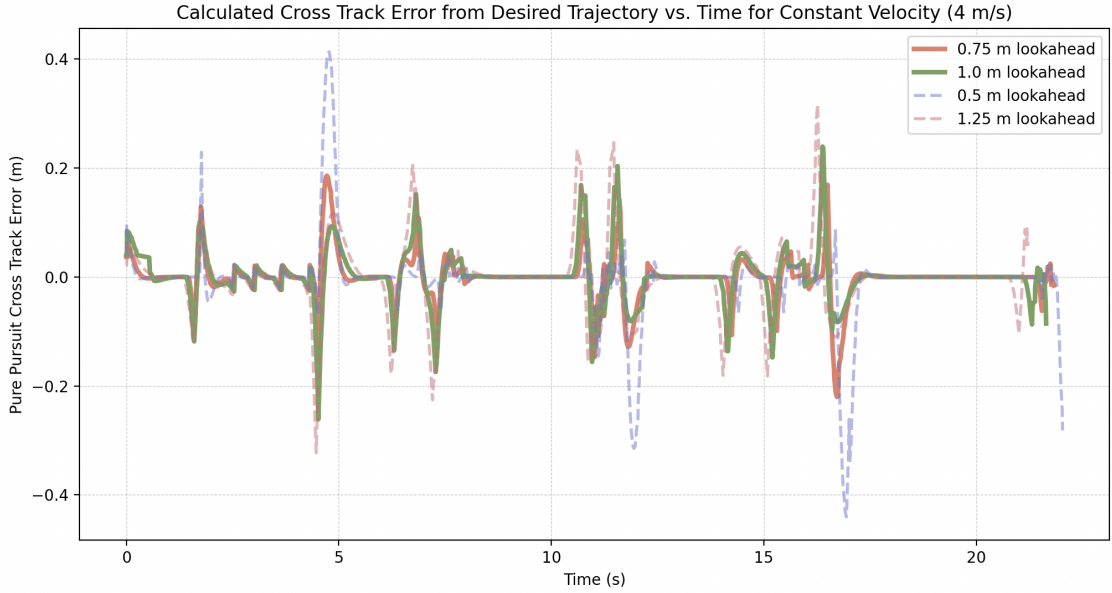


Figure 16: **Cross-track error over time.** Peaks correspond to turns; flat regions represent straight-line travel. Most turns generate two peaks: an entry deviation and an exit correction.

The dual peak pattern in cross-track error further supports these behaviors:

- **Low Lookahead (0.5 m):** Small entry spike, followed by a large correction at the exit.
- **High Lookahead (1.25 m):** Large initial deviation with smoother exits.
- **Medium Lookahead (0.75 m, 1.0 m):** Balanced behavior with symmetric spikes, indicating smooth corrections.

The tracking performance is summarized in Table X. After analysis, we selected 0.75 m and 1.0 m lookahead distances for real-world testing, providing a good balance between responsiveness and stability, with the real hardware used to determine the maximum safe speed.

Table X: Cross-track error statistics at different lookahead distances over 5 trials and constant velocity (4 m/s)

Lookahead Distance (m)	Mean Error (m)	Mean Absolute Error (m)	Standard Deviation (m)
0.5	-0.00358	0.02447	0.07335
0.75	-0.00138	0.01879	0.04009
1.0	-0.00003	0.02434	0.04599
1.25	-0.00012	0.03514	0.06304

3.6 Sim-to-Real Integration

For real-life experiments, we integrated our follower, planner, and localization stack, tuning the controller and dilation parameters for improved stability. While path planning and localization performed well, we revisited simulation lookahead values to see if their advantages held in practice. Using the same distances and speeds on the physical robot, we found that some simulation parameters didn't directly translate to real-world performance, as shown in Figure 19.

Environmental noise, such as objects obstructing LIDAR scans, introduced drift and inconsistencies in sensor data, causing instability at smaller lookaheads and oscillatory behavior. A larger lookahead reduced noise effects, resulting in smoother tracking.

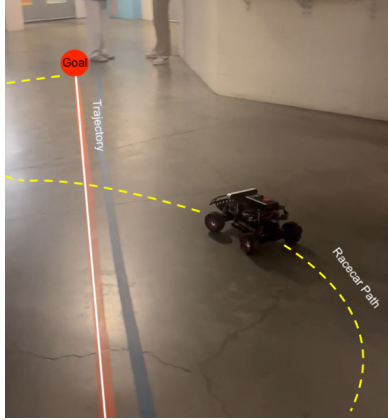
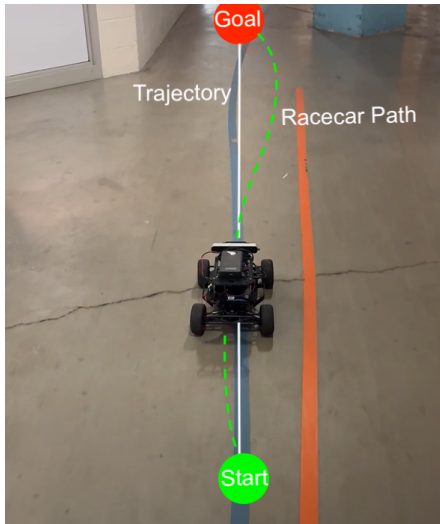
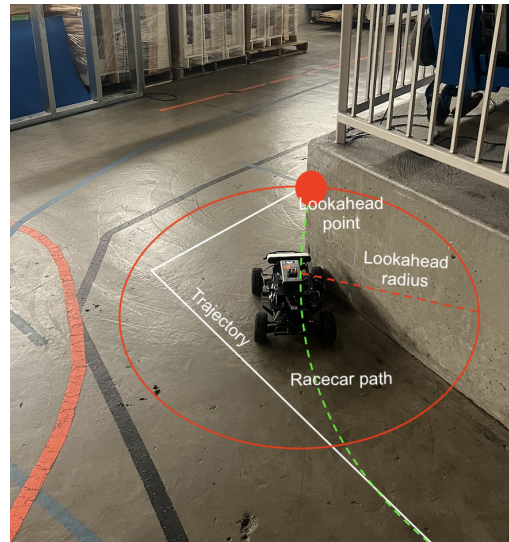


Figure 17: **Oscillatory behavior with small lookahead on real robot.** 0.75 m caused oscillations in real tests due to delayed steering and sensor noise, requiring larger lookahead distances.



(a) Larger lookahead (1.0 m) reduces oscillations on straight paths.



(b) Too large lookahead (1.25 m) causes corner cutting, leading to turn failures.

Figure 18: **Trade-offs of large lookahead distances.** 1.0 and 1.25 m lookahead smoothed straight-line steering, but 1.25 m caused turn failures, highlighting the challenge of balancing stability and accuracy.

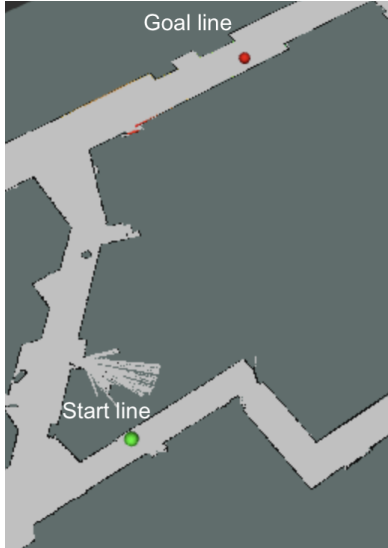


Figure 19: **Bonus navigation path and real-world performance.**

After testing our pipeline in the Stata basement, we completed the bonus path (Figure 16) in 38 seconds with a speed of 1.5 m/s and a lookahead of 1 m (Video) . Higher speeds caused collisions at a corner (Figure 18b), likely due to unmapped furniture interfering with LIDAR and affecting localization. Future work will adjust the Stata basement map and modify our A* algorithm to penalize driving too close to walls.

4 Conclusion: Sydney Matthews

The path planning lab successfully integrated our localization framework from the previous lab, along with path planning methods and a trajectory following controller, into a complete system for autonomous robot navigation. Our implementation of both a search-based (A*) and a sampling-based (RRT) planning algorithm enabled us to assess the trade-offs between computational efficiency and path optimality. We evaluated both algorithms quantitatively, and introduced a spline-based smoothing feature for qualitative observations of our path accuracy.

The pure pursuit controller we implemented provided effective, geometry-based route tracking, which depended on a set lookahead distance. This was carefully tuned both in simulation and on the real race car to find the perfect balance between under and overshooting. System integration through ROS2 allowed for modular development and robust testing in both simulation and hardware environments. Through comprehensive evaluation and parameter tuning, we demonstrated reliable performance across diverse navigation scenarios.

Future work will focus on addressing these challenges by improving our A* algorithm to penalize driving too close to walls, enhancing path smoothness, and developing methods to handle sensor occlusions. Additionally, we plan to integrate adaptive algorithms that can adjust to environmental changes in real-time. Further tuning of the lookahead distance will also be conducted to improve stability and reduce errors at higher speeds. As we continue to refine our system, we aim to extend the capabilities of our autonomous navigation framework to handle even more complex, real-world scenarios.

5 Lessons Learned

5.1 Pyae Sone Nyo Hmine

Making sure to log various parameters is very helpful in debugging. It's also important for us to connect concepts that we learned in the other labs and how to integrate them. As with other labs, we also found breaking the work into different modules and dividing and conquering to be efficient. We also noticed that it's important to not rely too much on sim as the sim to real gap can be quite large and it's important to start understanding what the differences are such as obstacles or friction in the wheels.

5.2 Shrika Eddula

The transition from simulation to physical hardware reinforced the challenges of real-world deployment for me, where sensor noise, computational constraints, and environmental variations require careful system calibration. I also learned the importance of parameter tuning in path planning systems, particularly lookahead distance. An important collaboration aspect I learned was starting work very early and working in parallel.

5.3 Sammy Krem

This lab taught me the importance of parameter tuning and clear communication in robotics. We saw firsthand how different lookahead distances affected performance. Smaller values improved turning accuracy but caused oscillations, while larger ones smoothed the path but led to corner cutting. Carefully tuning this parameter was essential for reliable tracking. I also learned how vital it is for everyone on the team to stay aligned. Miscommunication around deliverables slowed us down, but clear collaboration helped us debug effectively. Connecting lecture concepts to implementation and working together made a big difference in getting our algorithms to work smoothly.

5.4 Sydney Matthews

I learned how important parameter tuning is for getting everything to work smoothly, especially when moving from simulation to the real robot. Small changes to values made a big difference in how well things performed. I also saw how helpful it is to work on different parts of the project in parallel. Since we split up the modules early on, we were able to make faster progress and catch issues sooner when putting everything together. It made the integration process way more manageable.

5.5 Jayson Lin

In this lab, our solution was valid; however, it felt like the smallest parameter changes would lead to big errors. For instance, a 0.1 change in lookahead distance was the difference between strong oscillations and regular path following. Thus, I learned to be a lot more patient when isolating issues and not assuming the worst: that the implementation didn't work. I'm grateful for the patience my teammates demonstrated. With collaboration, it's infinitely easier to get past issues with uncertainties like this.